

Operating Systems Design

Course Code: CS207

Dr. Gunjan Chugh

Assistant Professor

**Dept. of Computer Science Engineering
Delhi Technological University**

Course Description

Unit No.	Content
1	Introduction: Operating system and function, Evolution of the operating system, Batch, Interactive, Time Sharing and Real-Time System, System protection. Operating System Structure: System Components, System Structure, Operating System Services.
2	Concurrent Processes: Process concept, Principle of Concurrency, Producer Consumer Problem, Critical Section problem, Semaphores, Classical problems in Concurrency, Inter Process Communication, Process Generation, Process Scheduling. CPU Scheduling: Scheduling Concept, Performance Criteria of Scheduling Algorithm, Evolution, Multiprocessor Scheduling.
3	Deadlock: System Model, Deadlock Characterization, Prevention, Avoidance and Detection, Recovery from deadlock combined approach.
4	Memory Management: Base machine, Resident monitor, Multiprogramming with fixed partition, Multiprogramming with variable partition, Multiple base register, Paging, Segmentation, Virtual memory concept, Demand paging, Performance, Paged replacement algorithms, Allocation of frames, Thrashing, Cache memory organization, Impact on performance.

Course Description

Unit No	Content
5	I/O Management & Disk Scheduling: I/O devices and organization of I/O function, I/O Buffering, DISK I/O, Operating System Design Issues. File System: File Concept, File Organization and Access Mechanism, File Directories, File Sharing, Implementation Issues.
6	Case Studies: Windows, Linux and Unix.

Recommended Reading Material

Text Books:

1. Silberschatz and Galvin, "Operating System Concepts", Pearson, 10th Ed
2. Tannenbaum, "Operating Systems", PHI, 4th Edition.

Reference Books:

1. Milenekovic, "Operating System Concepts", McGraw Hill
2. Dietel, "An introduction to operating system", Addison Wesley

Course Outcomes(CO)

- ❑ Understand the basic concepts, types of operating systems, and functions of operating systems.
- ❑ Analyze the basic concept of process and analyze different CPU scheduling algorithms
- ❑ Understand the basic concept of deadlock and its characteristics, and analyze various measures taken to prevent deadlock and for its recovery.
- ❑ Analyze various memory management schemes and their associated concepts like paging, segmentation, and virtual memory concepts.
- ❑ Understand and evaluate I/O management and File systems, including basic concepts of disk scheduling and Implementation Issues.
- ❑ Understand and Analyze Case Studies of Linux, Unix, and Windows

Evaluation Criteria

Components	Maximum Marks
CWS	15 (Attendance, Assignment/Quiz, Class Performance)
PRS	25 (Practical)
MTE	20 (Mid Term)
ETE	40 (End Term)
Total	100

Google Classroom

- The study material, Assignment, Quiz, and all other instructions related to the Course will be posted on Google Classroom.
- All students are requested to join the **Operating System, Section-A6** Google Classroom with the following Code:

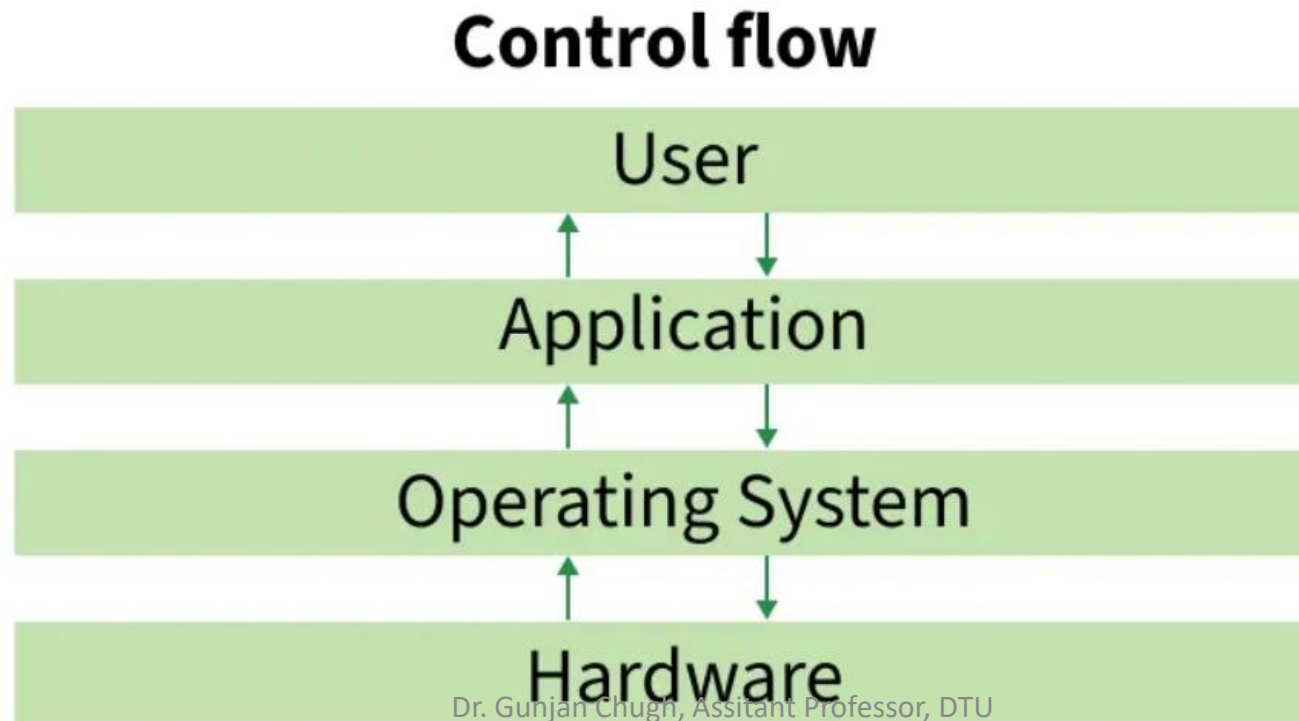
i5g4lax6

Contents of Module 1

- ❑ Introduction to Operating System
- ❑ Functions of OS
- ❑ Evolution of the operating system
- ❑ Batch, Interactive, Time Sharing and Real-Time System,
- ❑ System protection.
- ❑ Operating System Structure: System Components, System structure,
- ❑ Operating System Services.

What is an Operating System?

- ❑ An operating system **acts as an intermediary** between the user of a computer and the computer hardware.
- ❑ In short, it's an interface between computer hardware and the user.
- ❑ **The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.**



What is an Operating System?

❑ **Definition:** An operating system (OS) is software that manages computer hardware and software resources and provides common services for computer programs.

❑ **Goals:**

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Efficiently use the computer hardware.

What is an Operating System?

❑ OS is a resource allocator

- Manages all resources
- Decides between conflicting requests for efficient and fair resource use

❑ OS is a control program

- Controls execution of programs to prevent errors and improper use of the computer

Operating System as a User Interface

- ❑ The operating system coordinates the use of the hardware among the various system programs and application programs for various users.
- ❑ It simply provides an environment within which other programs can do useful work.
- ❑ An OS is a package of some programs that runs on a computer, allowing it to perform efficiently.

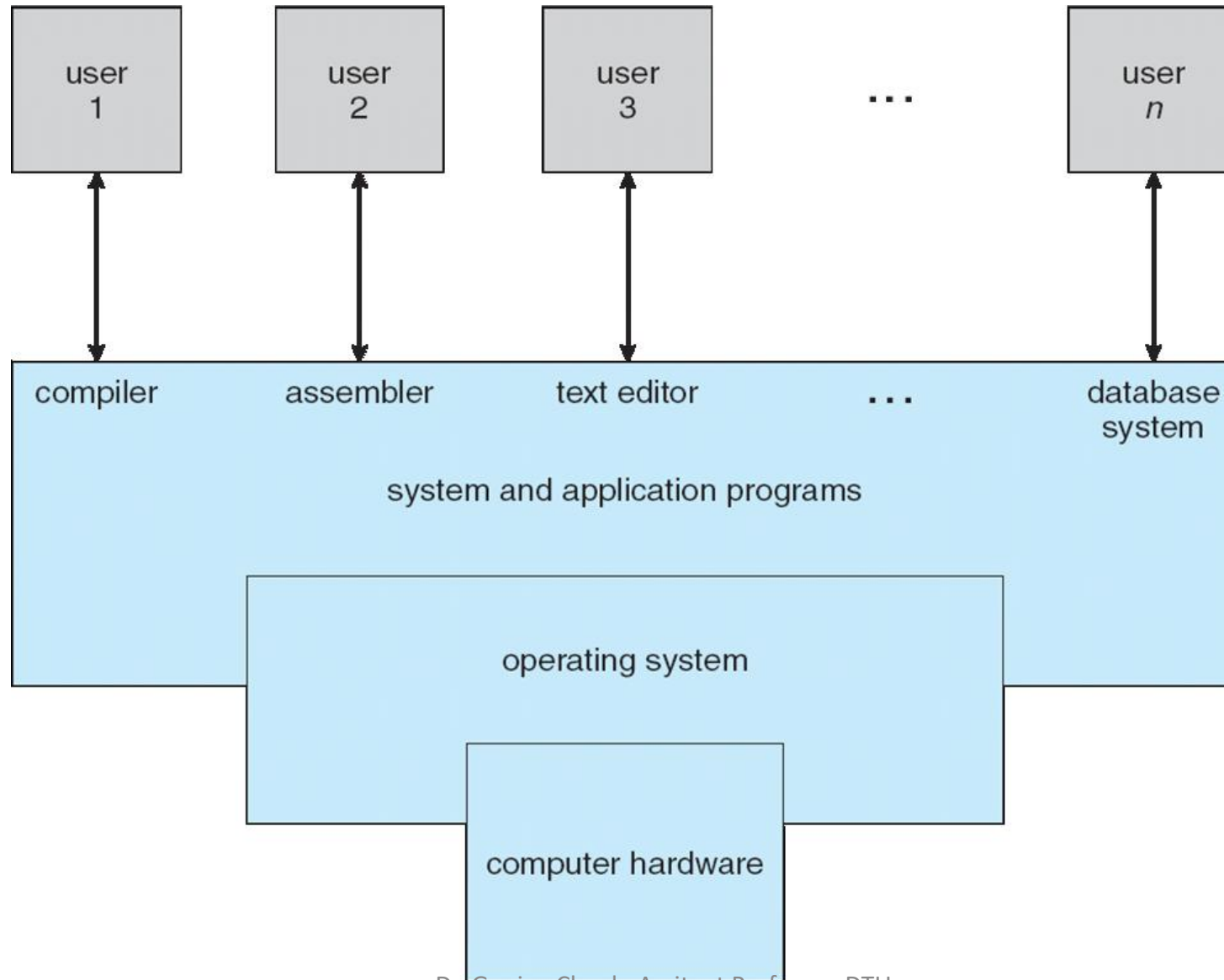
1.User (Interacts with the system)

2.System and application programs (Software tools for the user)

3.Operating system (Manages hardware and software resources)

4.Hardware (Physical computing devices)

Operating System as a User Interface



GOALS OF OPERATING SYSTEM

Primary Goals

The primary goals of an operating system (OS) are to provide a easy to use and convenient environment for executing user programs.

- ❑ **User Convenience:** It should be easy to use, providing a user-friendly interface and making it simple to interact with the system.
- ❑ **Program Execution:** It facilitates the execution of user programs, providing the necessary environment and services for them to run.
- ❑ **Resource Management:** The OS manages and allocates the computer's resources, including the CPU, memory, disk storage, and input/output devices, to ensure fair utilization.
- ❑ **Security:** The OS protects the system and user data from unauthorized access, ensuring the confidentiality, integrity, and availability of information.

GOALS OF OPERATING SYSTEM

Secondary Goals

- ❑ **Efficient Resource Utilization:** It should aim to maximize the performance and utilization of computer resources like CPU, Memory, and IO devices, ensuring that the system runs smoothly and efficiently.
- ❑ **Reliability:** It should be robust and reliable, able to handle errors and exceptions gracefully, ensuring that the system continues to operate smoothly. It should be modular in design and easy to debug.

COMMON OPERATING SYSTEM



Windows



macOS



Linux

Examples of OS



Android

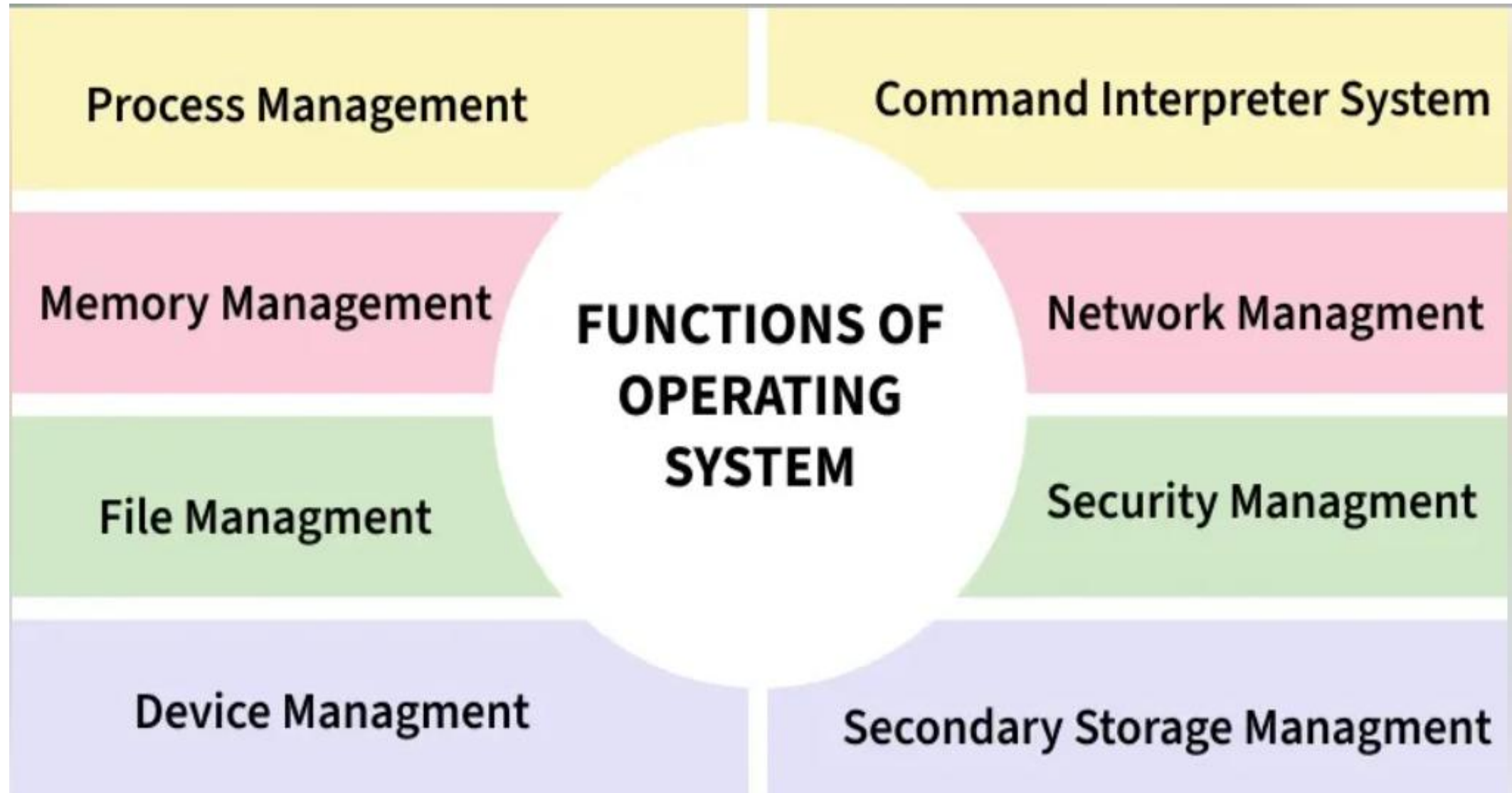


IOS

UNIX[®]
An Open Group Standard

UNIX

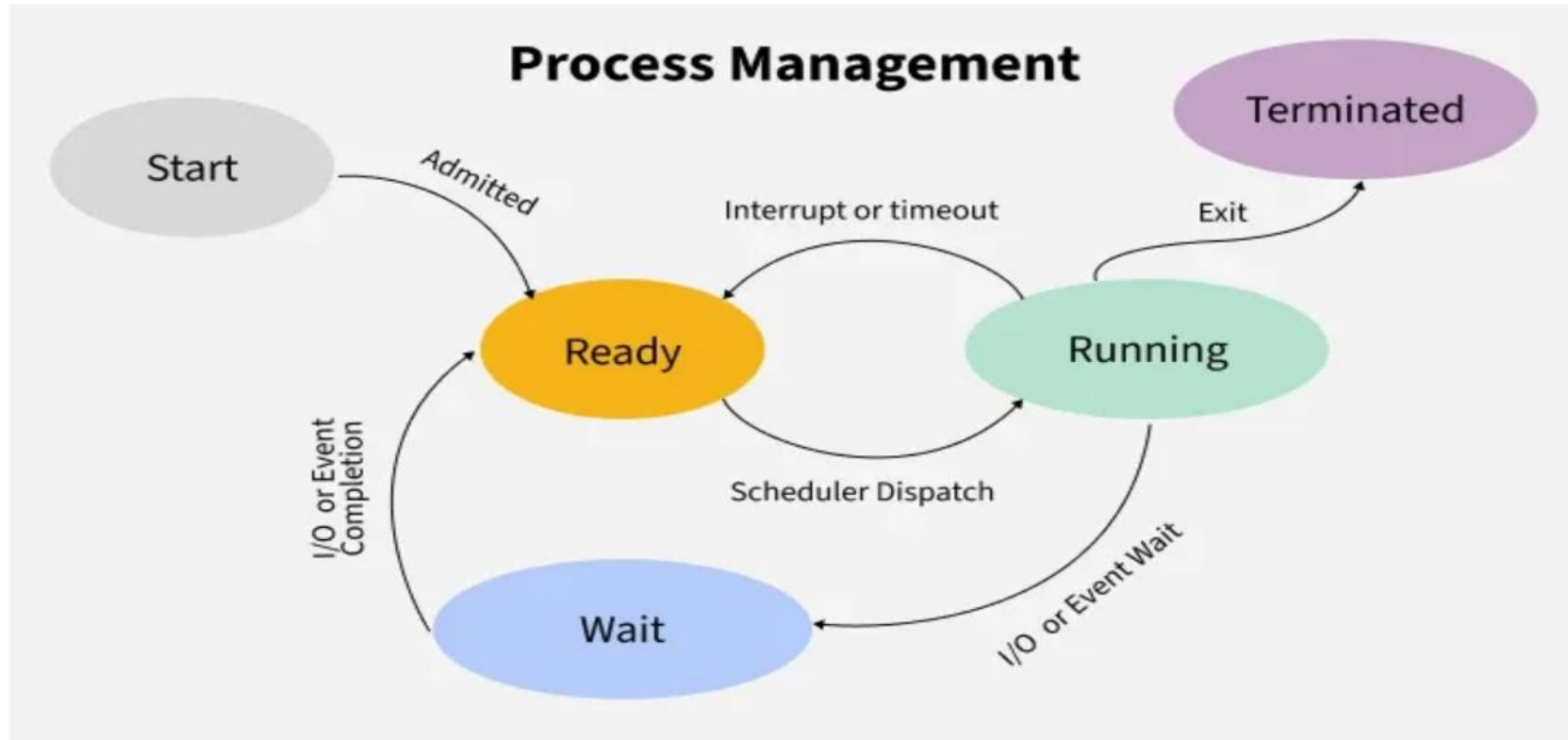
Functions of Operating System



1. Process Management

- ✓ A Process is a running program.
- ✓ The life cycle of a process is from the moment the program starts until it finishes.
- ✓ **The operating system makes sure each process:**
 - gets its turn to use the CPU
 - synchronized when needed
 - has access to the resources it needs, like memory, files, and input/output devices.
- ✓ It also handles issues like process coordination and communication, while preventing conflicts such as deadlocks.
- ✓ This way, the OS ensures smooth multitasking and efficient resource use.

1. Process Management



Core Functions in Process Management:

- **Process Scheduling**
- **Multitasking**
- **Process Synchronization**
- **Deadlock Handling**

2. Memory Management

Main Memory Management

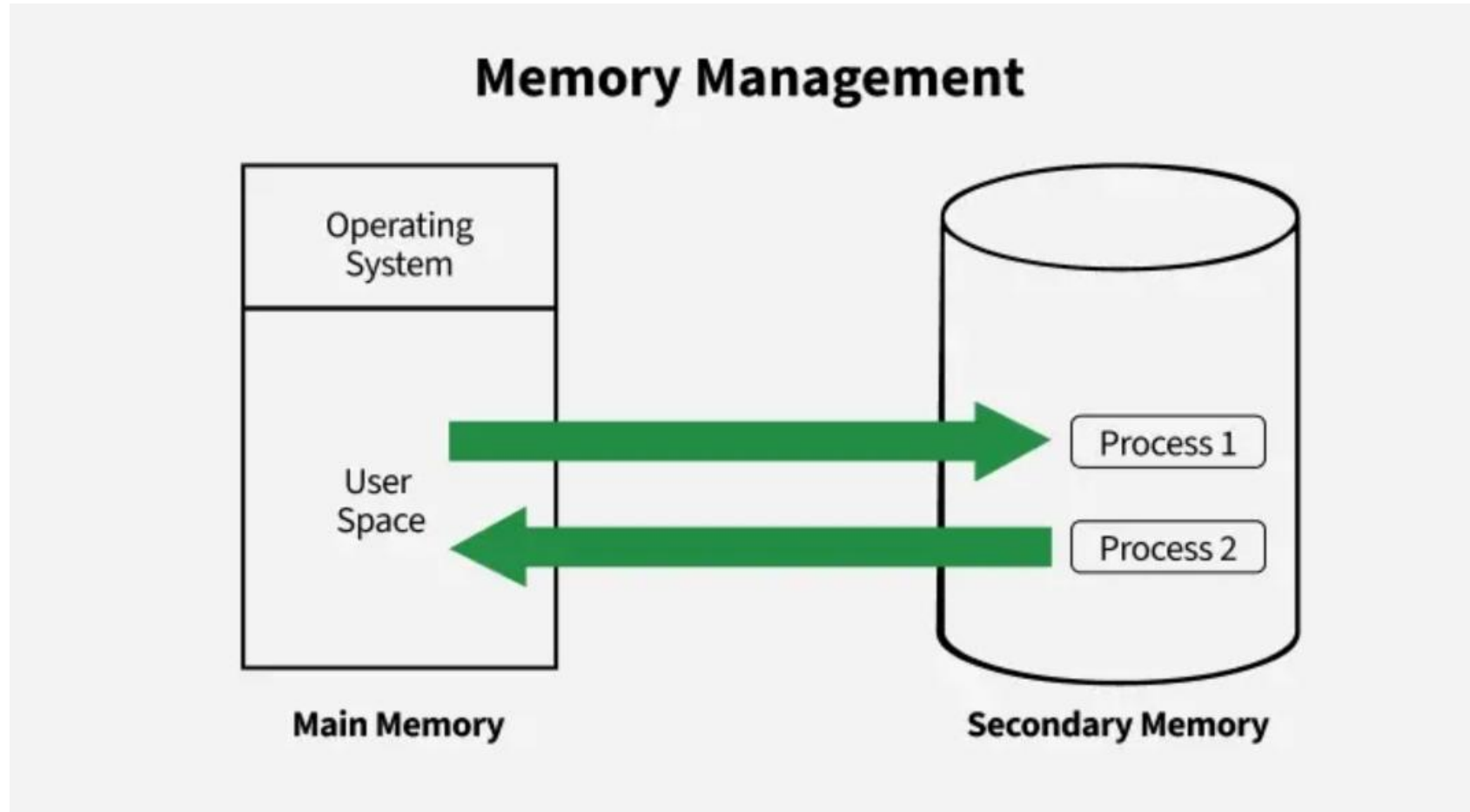
- **Memory Allocation:** Assigns memory to processes using techniques like paging and segmentation.
- **Memory Deallocation:** Frees memory when no longer needed.
- **Memory Protection:** Prevents processes from accessing each other's memory.
- **Virtual Memory:** Uses disk space as extra memory to run larger processes.
- **Fragmentation:** Manages wasted memory space (internal/external) through compaction.

2. Memory Management

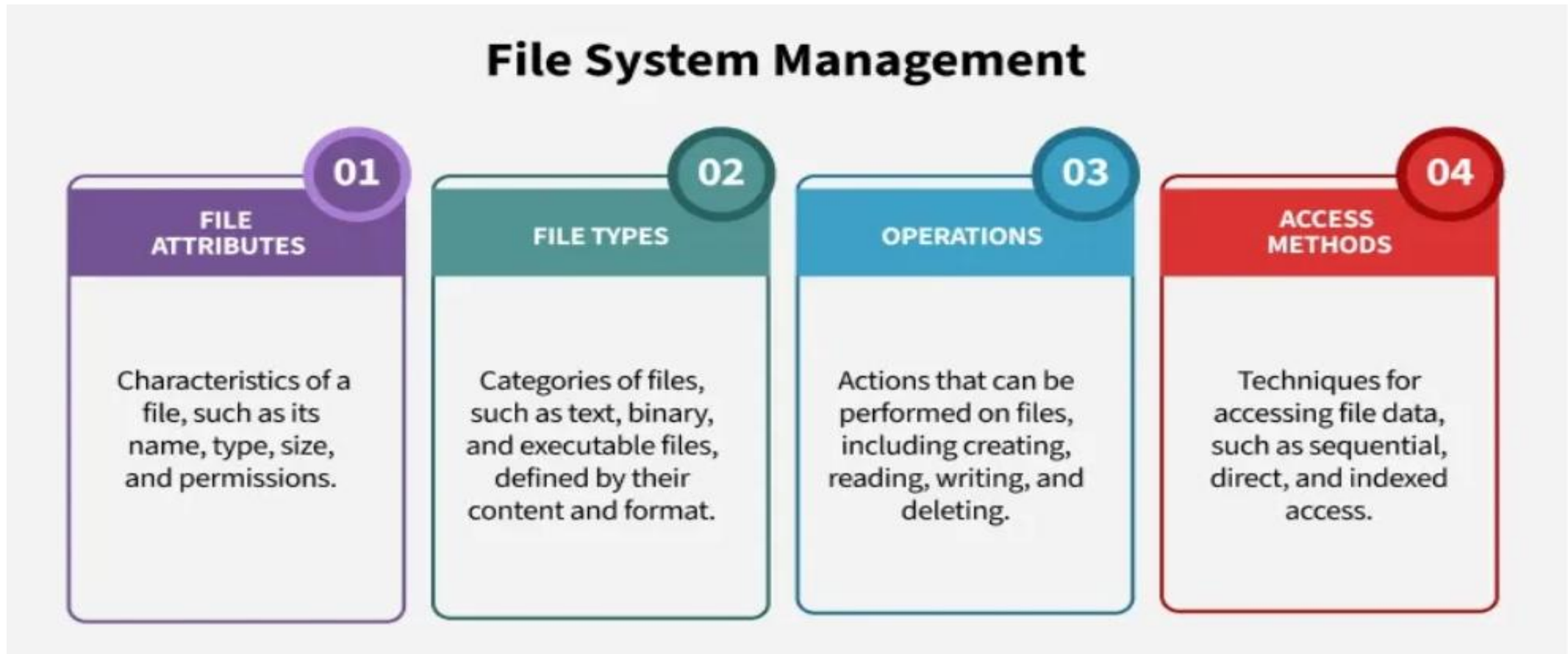
Secondary Memory Management

- **Disk Space Allocation:** Organizes how files are stored on the disk (contiguous, linked, indexed).
- **File System Management:** Manages files and directories for efficient data access.
- **Free Space Management:** Tracks available space on the disk.
- **Disk Scheduling:** Organizes the order of disk read/write requests.
- **Backup and Recovery:** Ensures data is backed up and can be restored after failure.

2. Memory Management



3. File System Management



3. File System Management

- ✓ **File management in the operating system ensures the organized storage, access, and control of files.**
- ✓ **File System Management includes managing:**
 - ✓ **File Attributes**
 - **File Name:** Identifies the file with a name and extension (e.g., .txt, .jpg).
 - **File Type:** Defines the format of the file (e.g., text, image, executable).
 - **Size:** The amount of storage the file occupies.
 - **Permissions:** Determines who can read, write, or execute the file.
 - ✓ **File Types**
 - **Text Files:** Contain human-readable content (e.g., .txt, .md).
 - **Binary Files:** Store data in binary format (e.g., .jpg, .mp3).
 - **Executable Files:** Contain program code (e.g., .exe, .out).

3. File System Management

✓ Operations on Files

- **Create:** Allows users to create new files.
- **Read:** Opens files to read their contents.
- **Write:** Modifies the contents of a file.
- **Delete:** Removes a file from the system.

✓ Access Methods

- **Sequential Access:** Reads data in order, from start to finish.
- **Direct Access:** Jumps to a specific part of the file.
- **Indexed Access:** Uses an index for quick data retrieval.

4. Device Management

- ✓ Device management of an operating system handles the communication between the system and its hardware devices, like printers, disks, or network interfaces.
- ✓ **Major components in Device Management:**
 - **Device Drivers:** The operating system uses **device drivers** to interact with hardware devices. There are two types of device drivers:
 - **Kernel-space drivers** run in the OS kernel, offering direct access to hardware.
 - **User-space drivers** run outside the kernel and are more isolated, providing safety but less performance.

4. Device Management

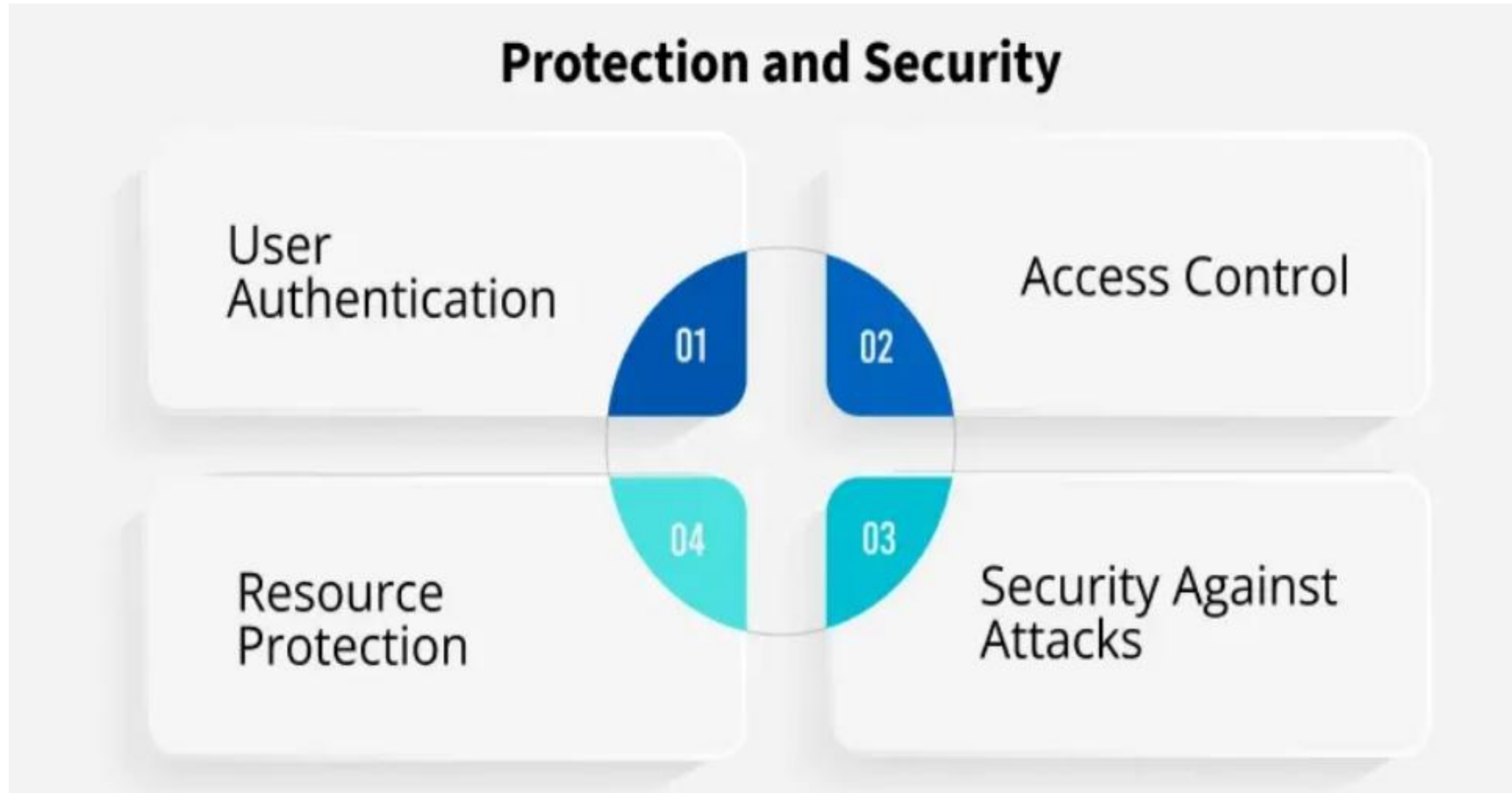
✓ Buffering & Caching:

- **Buffering temporarily stores data in memory to manage differences in device speeds.**
- **Caching improves access speed by storing frequently accessed data in a faster storage medium (like RAM).**

✓ Spooling:

- **Spooling manages data waiting to be processed, particularly in devices like printers.** The OS places print jobs in a spool (a temporary storage area), allowing the CPU to continue other tasks while the printer works through the queue.
- Other examples include mail spooling (for managing outgoing email) and batch-job spooling (for managing scheduled tasks).

5. Security and Protection Management



5. Security and Protection Management

- ✓ Ensures system security by managing user access and protecting against unauthorized access.
- ✓ Provides the following services:
 - **Access Control:** The operating system ensures that processes and users can only access resources they are authorized to.
 - **User Authentication:** A system identifies users through user IDs (UIDs) or Security IDs (SIDs).
 - **Resource Protection:** Mechanisms like file protection (ensuring only authorized users can access or modify files) and device protection (restricting direct access to device-control registers) ensure the integrity of system resources, preventing unauthorized or harmful use.
 - **Security Against Attacks:** Security mechanisms defend against external threats like viruses, worms, denial-of-service attacks, and identity theft. These attacks can misuse system resources, steal sensitive data, or disrupt system operation.

6. User Interface Management

- Provides a way for users to interact with the computer, either through
 - ✓ Command Line Interface(CLI)
 - ✓ Graphical User Interface (GUI)

Evolution of Operating System

❑ First Generation (1940s-1950s):

➤ Batch Processing Systems:

- ✓ No direct user interaction.
- ✓ Jobs were collected, grouped, and processed in batches.
- ✓ Example: IBM 701.

❑ Second Generation (1960s):

➤ Multiprogramming Systems:

- ✓ Several programs loaded into memory and executed one after another.
- ✓ Increased CPU utilization by organizing jobs so that the CPU always had one to execute.
- ✓ Example: IBM System/360.

Evolution of Operating System

❑ Third Generation (1970s):

➤ Time-Sharing Systems:

- ✓ Multiple users could interact with the computer at the same time.
- ✓ Led to the development of terminals and interactive sessions.
- ✓ Example: UNIX.

❑ Fourth Generation (1980s-1990s):

➤ Personal Computing:

- ✓ Emergence of personal computers (PCs).
- ✓ Graphical User Interface (GUI) became popular.
- ✓ Examples: MS-DOS, Windows 3.1, Mac OS.

Evolution of Operating System

❑ Fifth Generation (2000s-Present):

➤ Modern Operating Systems:

- ✓ Feature-rich OS with multitasking, multimedia support, and enhanced security.
- ✓ Increased focus on mobile and embedded systems.
- ✓ Mobile OSs like iOS (2007) and Android (2008) dominate.
- ✓ Cloud-based and virtualization technologies reshape computing
- ✓ Examples: Windows 10, macOS, Linux, Android, iOS.

Evolution of Operating System

❑ AI Integration - (Ongoing)

- ✓ With the growth of time, Artificial intelligence came into the picture.
- ✓ Operating system integrates features of AI technology like Siri, Google Assistant, and Alexa and have become more powerful and efficient in many ways.
- ✓ These AI features with the operating system create an entirely new feature like voice commands, predictive text, and personalized recommendations.

Types of Operating Systems

- ☐ Batch Operating Systems
- ☐ Time-Sharing Operating Systems
- ☐ Multi-Tasking Operating Systems
- ☐ Multi-Programming Operating Systems
- ☐ Multi-Processing Operating Systems
- ☐ Distributed Operating Systems
- ☐ Network Operating Systems
- ☐ Real-Time Operating Systems

Batch Operating System

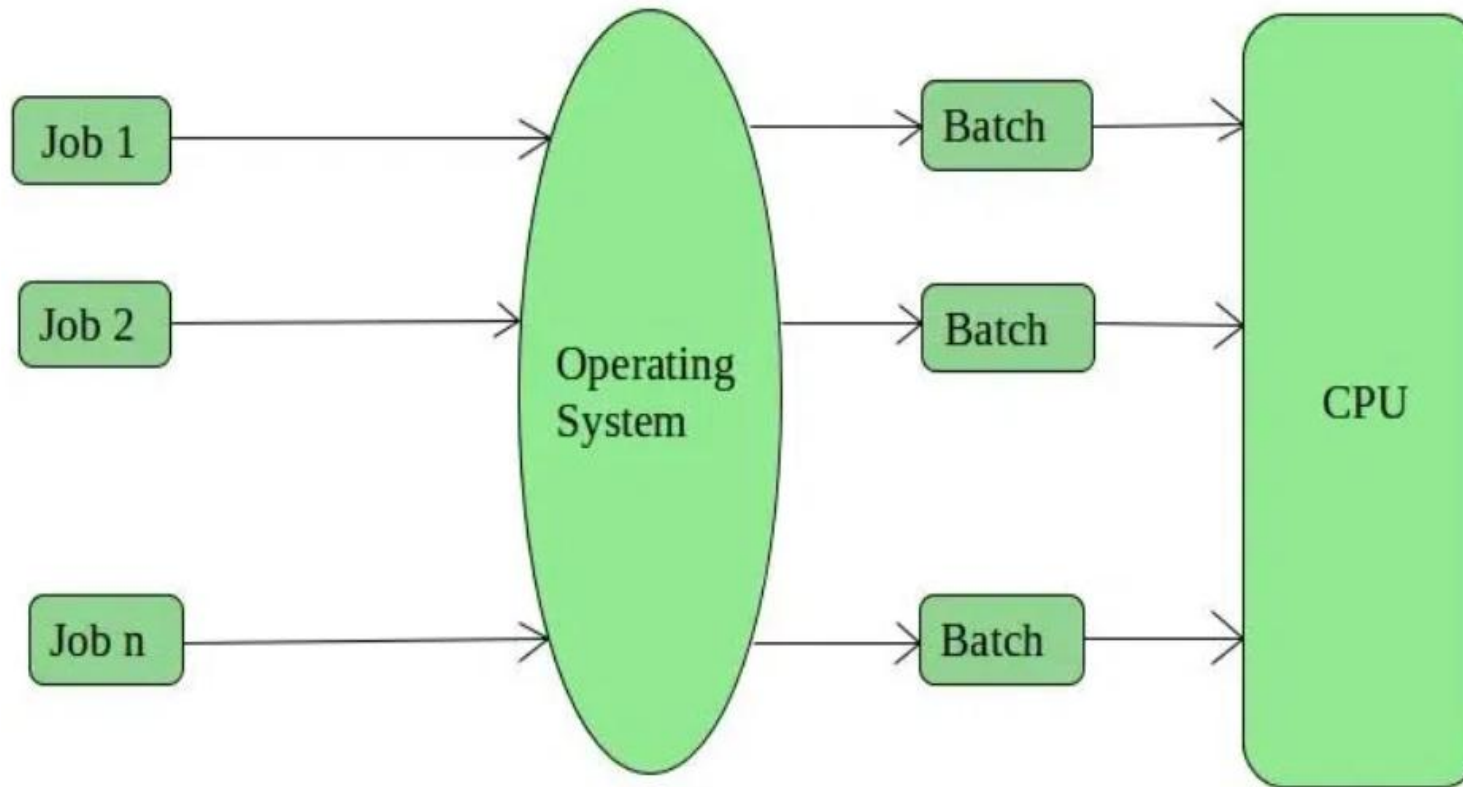
❑ Definition

- Executes a series of jobs without manual intervention.

❑ Characteristics

- These jobs are queued and executed one after the other, without user interaction during the process.
- Jobs processed in batches.
- No interaction with users during execution.

Batch Operating System



Batch Operating System

❑ Example

- Early IBM systems, punch card systems, Payroll system, Bank System

Batch Operating System

❑ Advantages

- Simple and efficient for large repetitive jobs.
- Efficient Job Management
- Minimal Idle Time
- Handling Repetitive Tasks

❑ Disadvantages

- No user interaction, leading to potential delays.
- Inefficient CPU Utilization
- Unpredictable Job Completion
- Increased Response Time
- Lack of Real-Time Feedback

Time Sharing Operating System

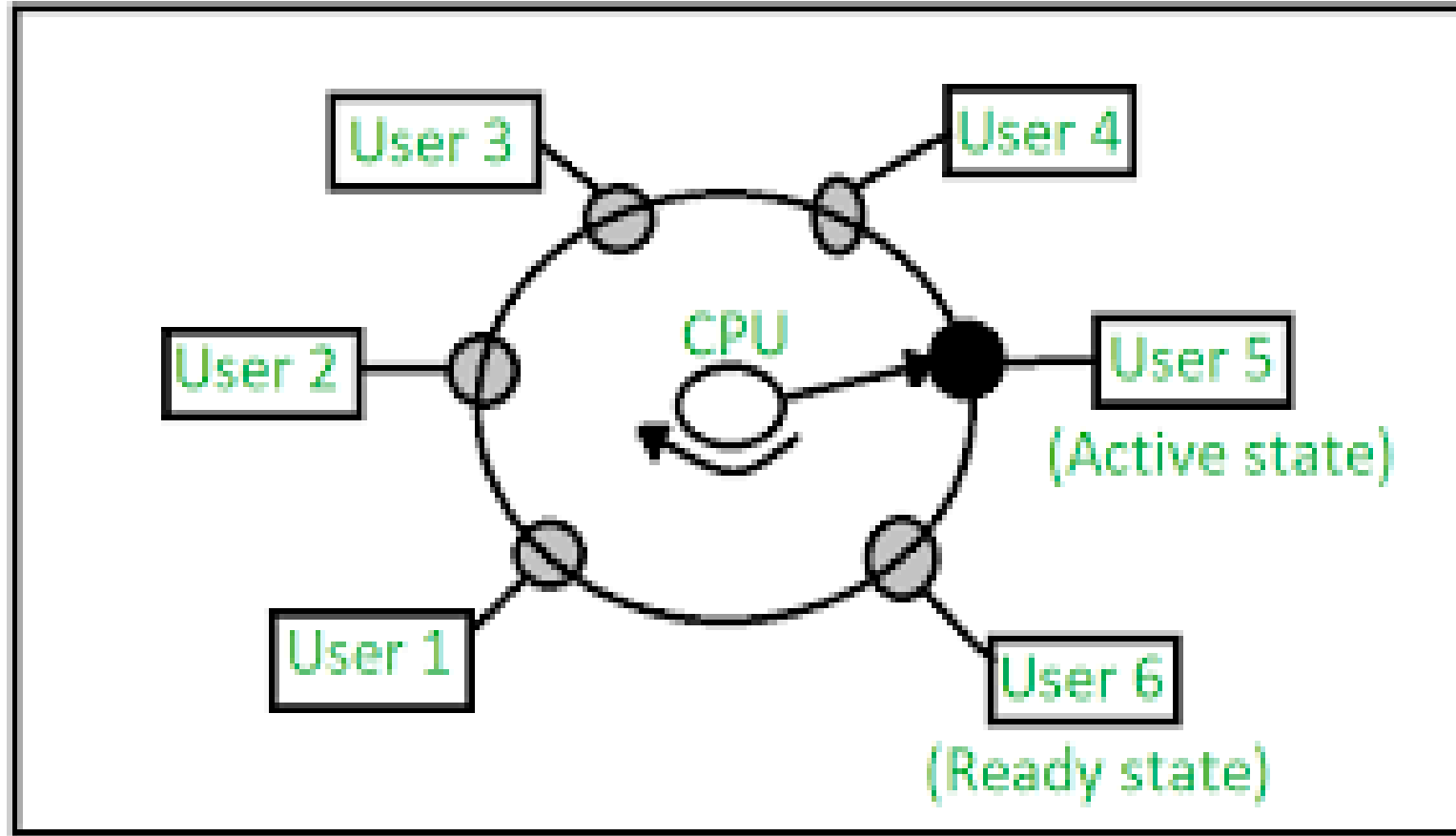
❑ Definition

- Multiple users can use the system simultaneously by sharing time slices of the CPU.

❑ Characteristics

- CPU time is divided among users, providing quick response.
- Supports multi-tasking and multi-user environments.

Time Sharing Operating System



In the above figure, user 5 is in an active state , but user 1, user 2, user 3, and user 4 are in a waiting state , whereas user 6 is in a **ready state**.

Time Sharing Operating System

❑ Advantages

- Increased Efficiency
- Cost-Effective
- Interactive Experience
- Resource Optimization

❑ Disadvantages

- Overhead due to frequent switching of tasks

❑ Example

- UNIX, Multics, Mainframe systems, etc.
- CTSS (Compatible Time-Sharing System): Developed at MIT, one of the first widely used time-sharing systems.

Multi-Tasking Operating System

❑ Definition

- Allows multiple tasks/processes to run concurrently.

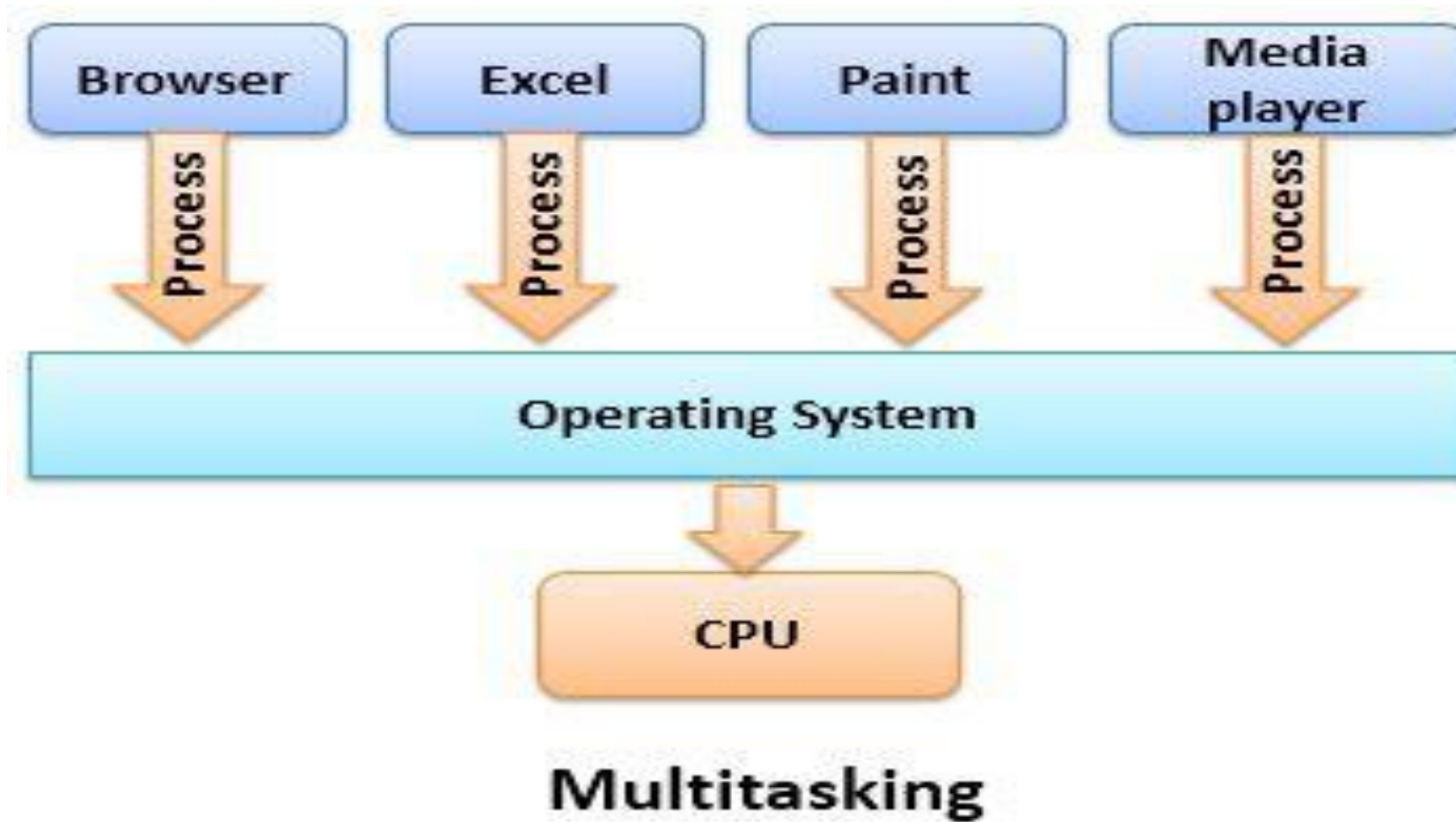
❑ Characteristics

- Single User with multiple tasks.
- CPU switches between tasks quickly, appearing simultaneous.
- Efficient use of CPU resources

❑ Example

- Windows, macOS, Linux.

Multi-Tasking Operating System



Advantages of Multitasking Operating Systems

- Increased Productivity:**

Users can work on multiple tasks at the same time, improving overall efficiency.

- Resource Sharing:**

Multiple users can access the same resources (like storage and processing power) simultaneously, optimizing system utilization.

- Improved Responsiveness:**

The system can switch quickly between tasks, making it feel more responsive.

- Efficient Resource Utilization**

- Background Processing:**

- Supports Multiple Users**

Disadvantages of Multitasking Operating Systems

- Increased Complexity:**

Multitasking operating systems are inherently more complex to design and implement than single-tasking systems.

- Higher Hardware Requirements:**

Multitasking often requires more powerful hardware to maintain acceptable performance.

- Overhead:**

Switching between tasks (context switching) requires overhead, which can slow down the system, especially with a large number of tasks.

Multi-Programming Operating System

❑ Definition

- Allows multiple programs to run simultaneously by managing their execution.

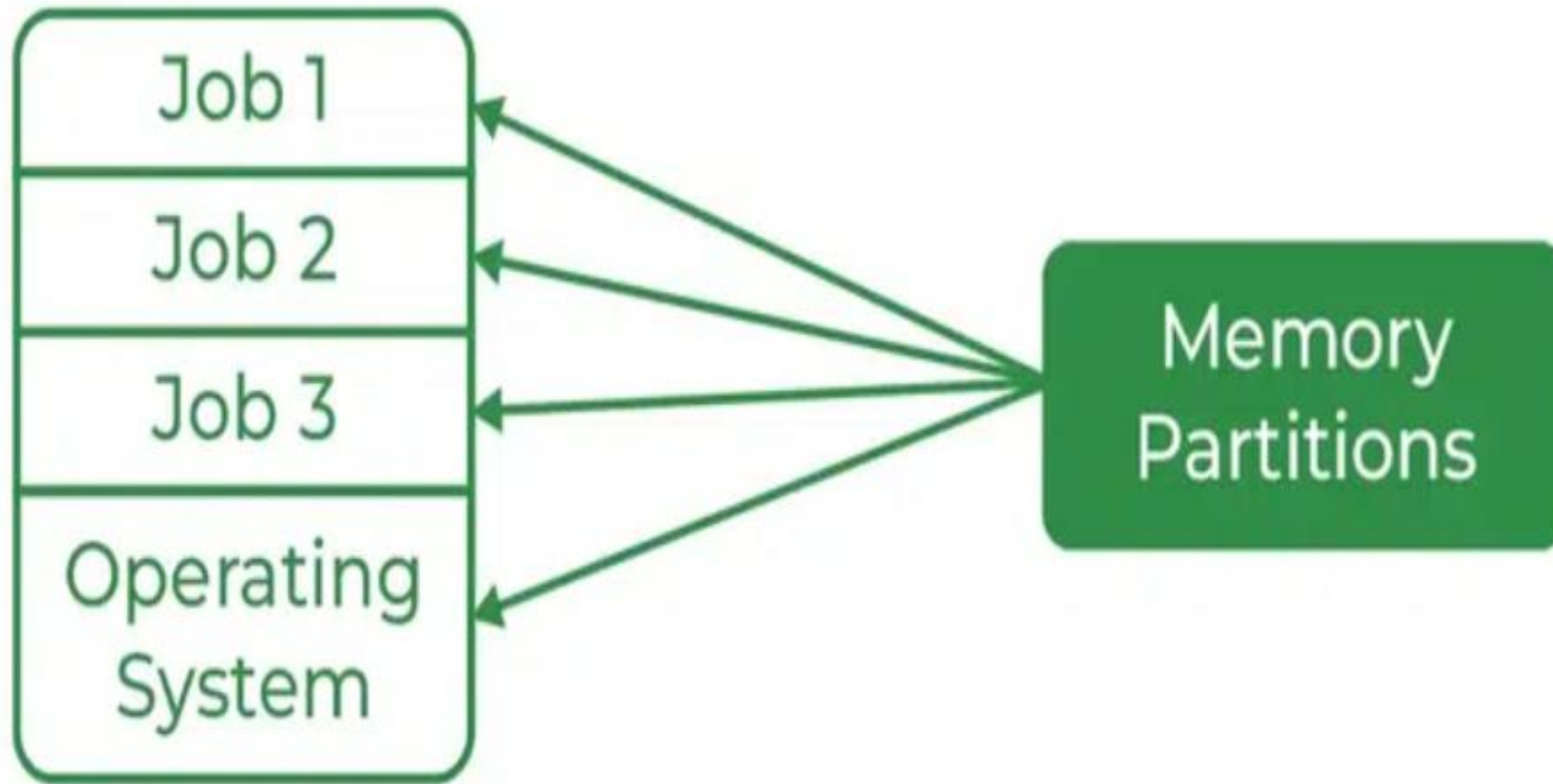
❑ Characteristics

- Based on I/O wait times.
- Increases CPU utilization by organizing jobs so the CPU always has one to execute

❑ Example

- Windows, Linux etc.

Multi-Programming Operating System



Multi-Programming Operating System

❑ Advantages

- CPU utilization is high because the CPU never goes to idle state.
- Memory utilization is efficient.
- CPU throughput is high and also supports multiple interactive user terminals.

❑ Disadvantages

- Complex memory management.

Multi-Processing Operating System

❑ Definition

- Supports the use of more than one processor to execute multiple tasks simultaneously.

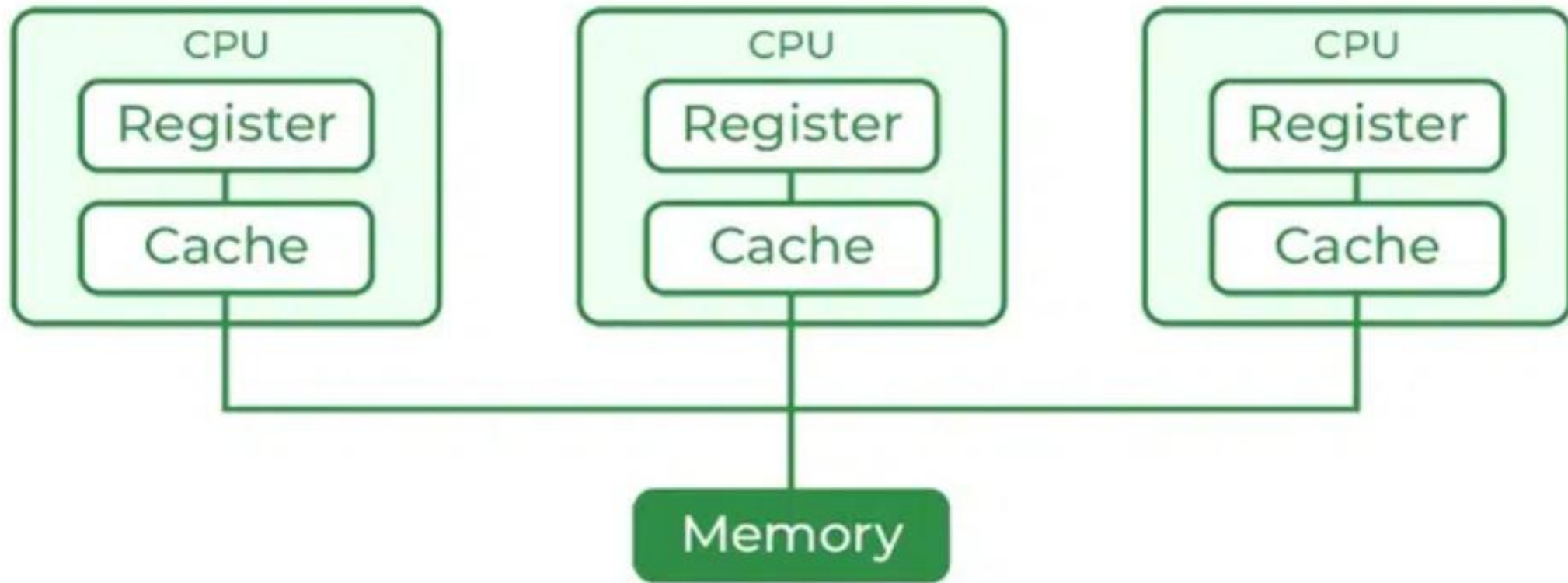
❑ Characteristics

- Parallel processing, increasing computational speed.
- Tasks divided among processors.

❑ Example

- IBM Mainframes, modern server systems, etc.

Multi-Processing Operating System



Multiprocessing Operating System

Multi-Processing Operating System

❑ Advantages

- Enhanced performance and reliability.
- Efficient handling of large, complex tasks.
- As it has several processors, so, if one processor fails, we can proceed with another processor.

❑ Disadvantages

- Complex to design and manage synchronization.

Real-Time Operating System

❑ Definition

- Processes data as it comes in, typically without buffering delays.

❑ Characteristics

- High reliability and performance.
- Used in systems with strict timing constraints.

❑ Purpose

- Unlike general-purpose operating systems (GPOS) like Windows or Linux, which are good at multitasking and handling various applications, a real-time operating system (RTOS) is **designed to manage time-sensitive tasks precisely.**
- **The main goal of an RTOS is to perform critical tasks on time.** It ensures that certain processes are finished within strict deadlines, making it perfect for situations where timing is very important.

Types of Real-Time Operating System

Hard Real-Time Operating System

- **These operating systems guarantee that critical tasks are completed within a range of time.** For example, a robot is hired to weld a car body.
- If the robot welds too early or too late, the car cannot be sold, so it is a hard real-time system that requires complete car welding by the robot hardly on time, scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Soft Real-Time Operating System

- **This operating system provides some relaxation in the time limit.** For example - Multimedia systems, digital audio systems, etc. Explicit, programmer-defined, and controlled processes are encountered in real-time systems.
- A separate process is changed by handling a single external event. The process is activated upon the occurrence of the related event signalled by an interrupt.

Real-Time Operating System

❑ Advantages

- Predictable response times, critical for real-time applications.

❑ Disadvantages

- Complex to design and implement.

❑ Example

- VxWorks, RTLinux, etc.

Networking Operating System

❑ Definition

- A **Network Operating System** manages a group of **computers connected over a network**, allowing them to **communicate, share files, printers**, and other resources.
- These systems run on a server and provide the capability to manage data, users, groups, security, applications and other networking functions.

❑ Characteristics

- Each computer runs **its own OS**
- **Users are aware** of the different systems on the network
- Manual resource sharing
- Suitable for **file servers, print servers**, office networks

Networking Operating System

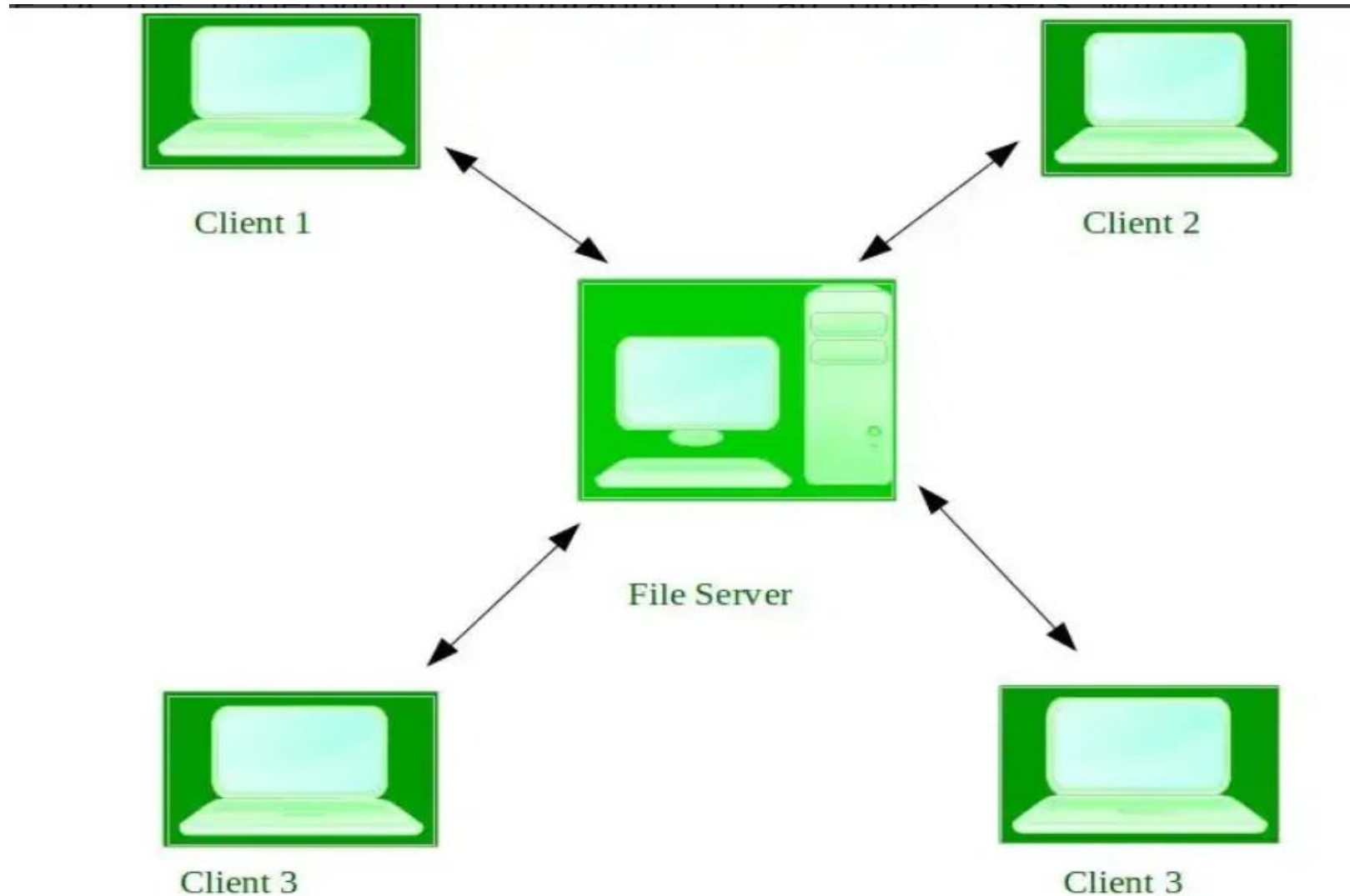
◆ Examples:

- Windows Server 2019
- Linux Ubuntu Server
- Novell NetWare
- macOS Server

◆ Real-Life Use:

In a school or office, All computers are connected to a server that handles logins and printers, but each computer operates independently

Networking Operating System



Networking Operating System

❑ Advantages

- Simplifies management of networked resources.
- Each computer operates on its own OS and can work independently
- Less expensive compared to distributed systems

❑ Disadvantages

- Limited transparency for users regarding resource distribution.
- Requires manual intervention for sharing and accessing resources

Distributed Operating System

❑ Definition

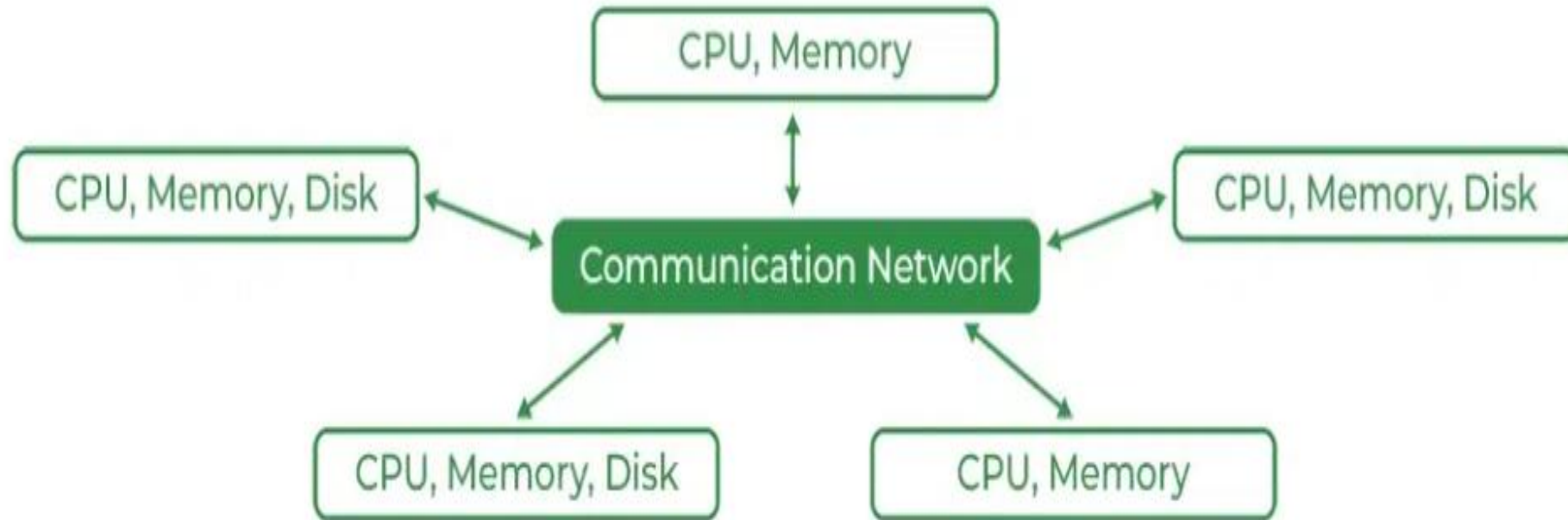
- A Distributed Operating System is an OS that manages a collection of separate computers and makes them appear to the users as a single unified system.

❑ Characteristics

- **Transparency** (Users feel like they're using one system)
- **Resource Management** – Handles CPU, memory, and file resources across multiple systems.
- **Scalability** – Can handle more machines being added.
- **Fault Tolerance** – If one node fails, others can take over.

Distributed Operating System

Architecture of Distributed OS



Distributed OS

Distributed Operating System

❑ Advantages

- High availability, scalability, and fault tolerance.
- Workload is distributed efficiently among available systems.
- Multiple systems working together ensure better uptime

❑ Disadvantages

- Complex to implement and manage.
- High cost of setup, hardware, and maintenance
- Errors may be hard to trace due to system complexity

❑ Example

- Google File System (GFS)
- Hadoop Distributed File System (HDFS)
- Amoeba, LOCUS, Plan 9

Real-Life Use:

- In cloud services or big data platforms (like Google or Facebook), multiple servers work together behind the scenes as if they are one big computer.

Mobile Operating System

- ❑ A **Mobile Operating System (Mobile OS)** is a specialized operating system designed to run on mobile devices such as smartphones, tablets, wearables, and other portable gadgets. It acts as the interface between the device's hardware and the user, managing resources and enabling applications to run

Key Features of a Mobile Operating System

1. User Interface (UI)
2. Multitasking
3. Connectivity
4. Application Management
5. Resource Management

Mobile Operating System

Advantages

- Portable and lightweight.
- Optimized for battery usage.
- Supports touchscreen & gesture navigation.
- Wide range of applications and hardware compatibility.

Examples of Mobile OS Usage

- **Android:** Samsung Galaxy, OnePlus, Xiaomi phones.
- **iOS:** iPhones, iPads.
- **HarmonyOS:** Huawei smartphones, smart TVs, IoT devices(many devoice in China).
- **KaiOS:** JioPhone and other basic internet-enabled phones(lightweight mobile OS-limited capabilities).

Comparison: DOS vs NOS

Feature	Distributed Operating System (DOS)	Network Operating System (NOS)
System Image	Appears as one unified system	Appears as independent systems
Transparency	High – user doesn't know where a task is running	Low – user knows the location of resources
Resource Sharing	Automatic and transparent	Manual or semi-automated
Failure Management	Redistributes tasks automatically	Each system handles its own failure
Dependency	Tightly coupled systems	Loosely coupled systems
Scalability	Limited (complex integration)	Highly scalable
Cost & Complexity	Higher	Lower
Real-Life Example	Google File System (GFS): Used by Google to manage massive distributed storage across multiple servers Apache Hadoop Distributed File System (HDFS) is used in big data platforms Dr. Gunjan Chugh, Assitant Professor, DTU	Windows Server 2019: Used in offices, schools, hospitals for centralized login, file & print sharing Red Hat Enterprise Linux (RHEL) Server in data centers

Comparison: MultiTasking vs Time Sharing

Aspect	Multitasking	Time Sharing
Definition	The ability of an OS to run multiple tasks (programs/processes) at the same time	A type of multitasking where multiple users access the system simultaneously using time slices
Focus	Multiple tasks/programs for a single user	Multiple users working on the system at the same time
User Type	Single user system (like personal computers)	Multi-user system (like mainframes, servers)
Example	Editing a document while listening to music on a PC	Many users accessing a shared server to run programs or access files
CPU Sharing	CPU switches rapidly between processes	CPU time is divided into time slots and given to each user in turn
Response Time	Faster, as it's focused on one user	Slower, as system switches between users
Complexity	Less complex	More complex due to user isolation, scheduling, security

Difference between Multi Tasking OS and Multi Programming OS

Feature	Multi Programming OS	Multi Tasking OS
Primary Focus	Maximising CPU Utilisation	Concurrent Task Execution
Execution	Multiple programs in memory, one active at a time	Multiple tasks appear to run simultaneously
User Interaction	Typically less interactive, batch processing	Highly interactive, real-time response
CPU Switching	Based on I/O wait times	Rapid switching (time slices)
Memory Management	Complex, managing multiple programs in memory	Less complex, managing tasks within a single program session
Use Cases	Large-scale batch processing, mainframes	Personal computers, mobile devices
Examples	Early UNIX systems, IBM mainframes	Windows, macOS, Linux

System Protection

❑ Definition

- Mechanisms implemented in an operating system to control access to resources and data, ensuring system integrity and security.

❑ Objective

- Prevent Unauthorized Access:
 - ❖ Ensure only authorized users and processes can access system resources.
- Maintain Data Integrity:
 - ❖ Protect data from being altered or deleted by unauthorized entities.
- Ensure System Availability:
 - ❖ Prevent malicious activities that could disrupt system services and resources.
- Confidentiality:
 - ❖ Safeguard sensitive information from unauthorized disclosure.

Types of System Protection Mechanism

1. Authentication

- Authentication is the process of **verifying the identity** of a user, device, or system. It ensures that the entity requesting access is who it claims to be.

Purpose:

- Prevents **unauthorized access**
- Serves as the **first step** before authorization
- Builds **trust** between user and system

Types of System Protection Mechanism

Common Methods of Authentication

Method	Description
Passwords	Most common method; users provide a secret word or phrase.
Biometrics	Uses unique biological traits like fingerprint, facial recognition, retina scan, etc.
Multi-Factor Authentication (MFA)	Combines two or more methods (e.g., password + OTP sent to phone) for enhanced security.
Token-based Authentication	Uses a secure token or digital certificate to authenticate a session.
Smart Cards	Physical cards with embedded chips used for identity verification.

Types of System Protection Mechanism

2. Authorisation

- Authorization is the **process of determining what an authenticated user is allowed to do.**
- It defines **access levels** and **permissions** for resources or actions within a system.

Purpose:

- Controls **who** can **access, modify, or execute** certain parts of a system.
- Ensures **security, privacy, and data integrity.**

Types of System Protection Mechanism

Common Methods of Authorization

Method	Description
Access Control Lists (ACLs)	Each object (like file or resource) has a list specifying which users or system processes can access it and what operations they can perform (read, write, execute).
Role-Based Access Control (RBAC)	Permissions are assigned to roles (e.g., admin, editor, viewer), and users are assigned roles. Simplifies management of user permissions.
Attribute-Based Access Control (ABAC)	Permissions are based on attributes (e.g., user role, department, time of access, location).
Policy-Based Access Control (PBAC)	Uses policies defined by administrators to determine access based on a set of rules.

Types of System Protection Mechanism

Authentication vs Authorization

Aspect	Authentication	Authorization
What it does	Verifies identity	Grants access rights
Occurs	First	After authentication
Example	Logging in with username/password	Accessing admin panel based on user role

Types of System Protection Mechanism

3. Encryption

Definition:

Encryption is the process of converting **plaintext** into **ciphertext** (an unreadable format) to protect data from unauthorized access.

Purpose:

Ensures **confidentiality** and **data security** during storage or transmission.

Types:

Type	Description
Symmetric Encryption	Uses the same key for both encryption and decryption (e.g., AES, DES).
Asymmetric Encryption	Uses a public key for encryption and a private key for decryption (e.g., RSA, ECC).

Types of System Protection Mechanism

4. Firewall

A firewall monitors and controls incoming and outgoing network traffic based on predefined security rules.

Purpose:

Protects systems by blocking unauthorized access while permitting legitimate communication.

Types:

Type	Description
Hardware Firewall	A physical device placed between the internal network and external network (e.g., router firewall).
Software Firewall	Installed on individual systems to monitor and filter traffic (e.g., Windows Defender Firewall).

Types of System Protection Mechanism

5. Intrusion Detection System(IDS)

Definition:

An IDS monitors network or system activities for **suspicious behavior, malicious activities, or policy violations**.

Purpose:

Detects potential **intrusions** and sends **alerts** to administrators.

Types:

Type	Description
Network IDS (NIDS)	Monitors traffic across the entire network .
Host IDS (HIDS)	Monitors individual devices or hosts .

Types of System Protection Mechanism

6. Sandboxing

Definition:

Sandboxing isolates applications or code in a **controlled, restricted environment** to observe behavior without affecting the main system.

Purpose:

- Prevents malware or suspicious programs from **spreading or harming** the host system.
- Commonly used in **malware analysis** and **testing**.

Advantages of system protection

- Ensures the **security and integrity** of the system
- **Prevents unauthorized access**, misuse, or modification of the operating system and its resources
- Protects **sensitive data**
- Provides a **secure environment** for users and applications
- **Prevents malware** and other security threats from infecting the system
- Allows for **safe sharing of resources** and data among users and applications
- Helps maintain compliance with security regulations and standards

Disadvantages of system protection

- Can be **complex and difficult to implement and manage**.
- **May slow down system performance** due to increased security measures.
- Can cause **compatibility issues** with some applications or hardware.
- Can create a false sense of security if users are not properly educated on safe computing practices.
- Can create additional costs for implementing and maintaining security measures.

Operating system Structure

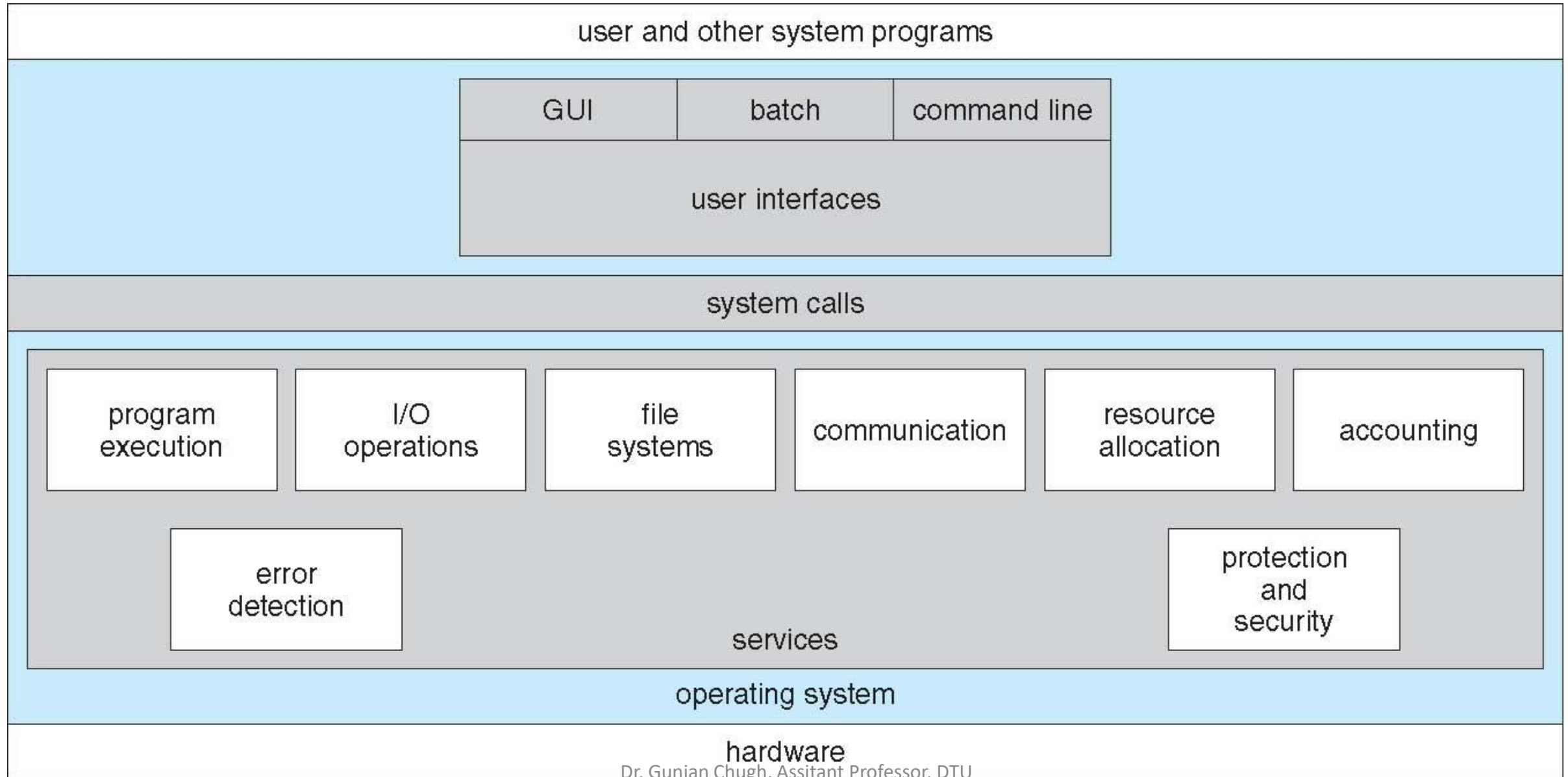
- The **structure of an Operating System (OS)** describes how its components are organized and interact to manage hardware, software, and resources efficiently.
- Different OS designs follow different structural models depending on performance, security, and flexibility goals
- The structure of the OS depends mainly on how the various standard components of the operating system are interconnected and merge into the kernel.

Types of Operating Systems Structures

- Simple Structure
- Layered Structure
- Modular Structure
- Virtual Machines
- Monolithic Structure
- Micro-Kernel Structure
- Hybrid-Kernel Structure
- Exo-Kernel Structure

All these structures are covered in a separate PDF File.

A View of Operating System Services



Operating System Services

❑ User Interface

- Almost all operating systems have a user interface (UI).
- Varies between Command-Line (CLI), Graphics User Interface (GUI).

❑ Program Execution

- The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating an error).

❑ I/O Operations

- A running program may require I/O, which may involve a file or an I/O device.

Operating System Services

❑ File System Manipulation

- The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, and manage permissions.

❑ Communication

- Processes may exchange information on the same computer or between computers over a network
- Communications may be via shared memory or through message passing (packets moved by the OS)

Operating System Services

❑ Error Detection

- OS needs to be constantly aware of possible errors
 - ✓ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ✓ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ✓ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.

❑ Accounting

- To keep track of which users use how much and what kinds of computer resources.

Operating System Services

❑ Resource Allocation

- When **multiple users or multiple jobs are running concurrently**, resources must be allocated to each of them.
- Many types of resources - CPU cycles, main memory, file storage, I/O devices.

❑ Protection and Security

- The owners of information stored in a multiuser or networked computer system may want to control its use; concurrent processes should not interfere with each other.
- **Protection:** it involves ensuring that all access to system resources is controlled.
- **Security:** It involves securing the system from outsiders for invalid access attempts.

Bootstrap Program

❑ **Definition:** For a computer to start running—for instance, when it is powered up or rebooted—it needs to have an initial program to run. This initial program is called **bootstrap program**

❑ **Characteristics**

- It is stored within the computer hardware in read-only memory (ROM) or electrically erasable programmable read-only memory (EEPROM), known by the general term **firmware**
- It initializes all aspects of the system, from CPU registers to device controllers to memory contents.
- The bootstrap program must know how to load the operating system and how to start executing that system.
- The bootstrap program must locate the operating-system kernel and load it into memory.
- Once the kernel is loaded and executing, it can start providing services to the system and its users.

Modes in Operating System

- ❑ To distinguish between execution of Operating System Code and User Defined Code, the Operating System works on two mode.
 - ❑ User Mode
 - ❑ Kernel Mode/ System Mode/ Supervisor Mode/Privileged Mode
- ❑ MODE BIT: To indicate the current mode, a bit is added to the hardware of the computer which is called **Mode Bit**
 - ❑ Kernel Mode: Mode bit is 0
 - ❑ User Mode: Mode bit is 1

Modes in Operating System

- ❑ At system boot time, the hardware starts in kernel mode.
- ❑ The operating system is then loaded and starts user applications in user mode.
- ❑ Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (that is, changes the state of the mode bit from 1 to 0).
- ❑ Thus, whenever the operating system gains control of the computer, it is in kernel mode.
- ❑ The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.

System Calls

- ❑ When a user application requires a service from the Operating System, it generates a **System Call**.
- ❑ The system must transition from User to Kernel Mode to fulfil the System call request.
- ❑ A system call is a way for a program (running in user mode) to request a service from the operating system's kernel (running in kernel mode).
- ❑ **They act as the interface between user programs and the OS.**

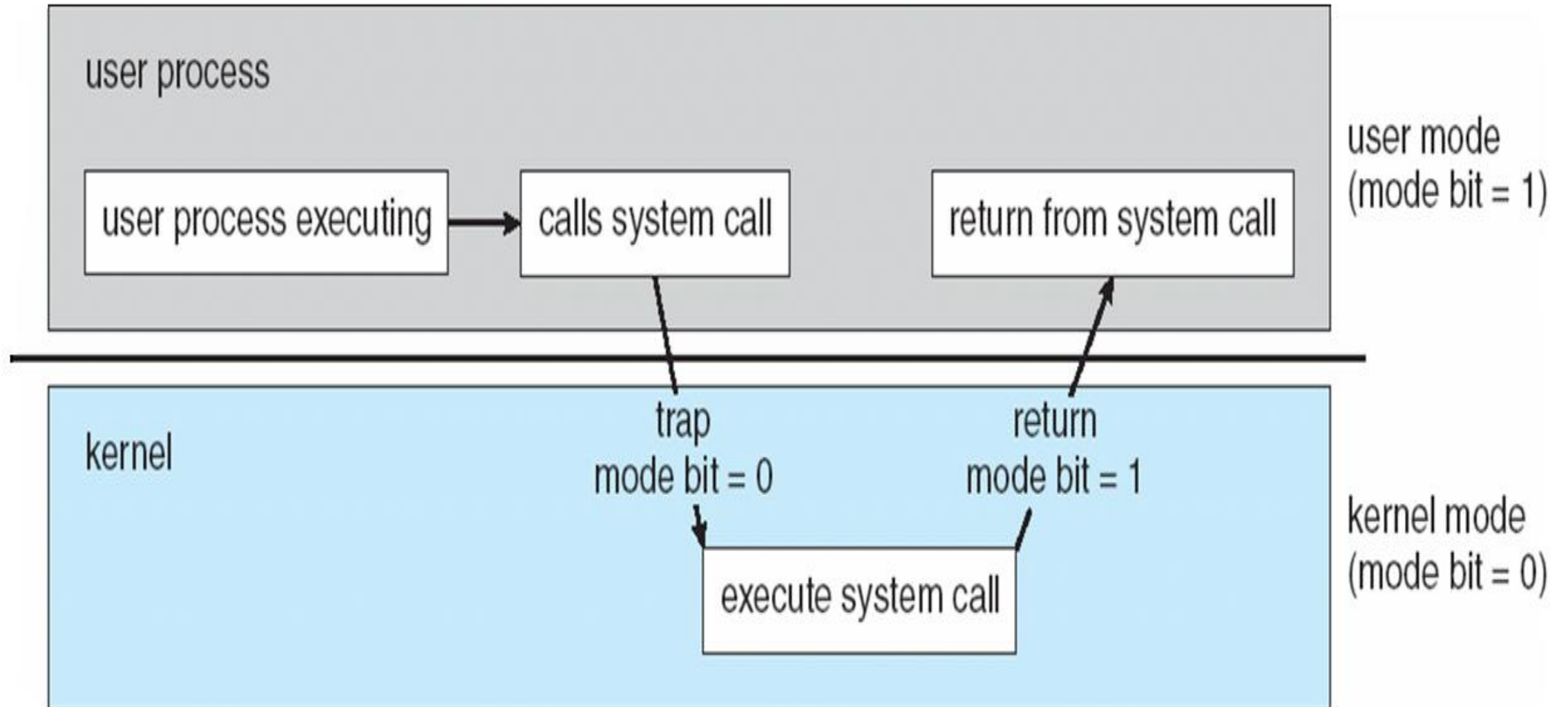
Why System Calls are Needed

- User programs can't directly access hardware (e.g., disks, printers, network) for safety and stability.
- System calls let programs ask the OS to perform tasks like file operations, process management, or network communication.

System Calls Working

1. User program makes a request (e.g., `read()`).
2. Request goes to the system call interface via a software interrupt or trap.
3. OS kernel executes the requested service.
4. The result is returned to the user program.

Working of System Calls



Types of System Calls

- **Process control**

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single-step** execution
- **Locks** for managing access to shared data between processes

Types of System Calls

- **File management**

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

- **Device management**

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

Types of System Calls

- **Information maintenance**

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

- **Communications**

- create, delete communication connection
- send, receive messages if **message passing model** to **host name** or **process name**
 - From **client** to **server**
- **Shared-memory model** create and gain access to memory regions
- transfer status information
- attach and detach remote devices

Types of System Calls

- **Protection**

- Control access to resources
- Get and set permissions
- Allow and deny user access

System Calls in Windows vs UNIX/LINUX

Function	Windows API / System Call	UNIX / LINUX
Process Creation	CreateProcess()	fork()
Execute Program	Part of CreateProcess() or LoadLibrary()	exec() family (execl, execv, etc.)
Process Termination	ExitProcess()	exit() / _exit()
Wait for Process	WaitForSingleObject()	wait() / waitpid()
Open File	CreateFile()	open()
Read File	ReadFile()	read()

Function	Windows API / System Call	UNIX / LINUX
Write File	WriteFile()	write()
Close File	CloseHandle()	close()
Get File Attributes	GetFileAttributes()	stat() / fstat()
Change File Attributes	SetFileAttributes()	chmod() / chown()
Delete File	DeleteFile()	unlink()
Create Directory	CreateDirectory()	mkdir()
Remove Directory	RemoveDirectory()	rmdir()
Get Current Time	GetSystemTime() / GetLocalTime()	time() / gettimeofday()
Sleep / Delay	Sleep()	sleep()
Memory Allocation	VirtualAlloc()	sbrk() / mmap()
Create Thread	CreateThread()	pthread_create()
Socket Creation	WSASocket() / socket()	socket()
Send Data (Network)	send()	send()
Receive Data (Network)	recv()	recv()
Shutdown System	ExitWindowsEx()	shutdown() / reboot()